

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение

высшего образования

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет программной инженерии и компьютерной техники

Образовательная программа: Системное и прикладное программное
обеспечение

Направление подготовки: Программная инженерия 09.04.04

Дисциплина: Системное программное обеспечение

Практическое задание №2

Обучающийся: Пименов Глеб Андреевич 506758

Группа: Р4115

Дата 20.01.2026

Санкт-Петербург

2026

Оглавление

1. Цели	2
2. Задачи	2
3. Описание работы	6
4. Аспекты реализации	6
4.1. Описание структур данных	6
4.2. Описание основного алгоритма построения графа потока управления	7
4.3. Описание некоторых функций	8
Результаты	15
Выводы	17

1. Цели

Реализовать построение графа потока управления посредством анализа дерева разбора для набора входных файлов. Выполнить анализ собранной информации и сформировать набор файлов с графическим представлением для результатов анализа.

2. Задачи

Порядок выполнения:

1. Описать структуры данных, необходимые для представления информации о наборе файлов, наборе подпрограмм и графе потока управления, где:
 - а. Для каждой подпрограммы: имя и информация о сигнатуре, граф потока управления, имя

исходного файла с текстом подпрограммы.

б. Для каждого узла в графе потока управления, представляющего собой базовый блок

алгоритма подпрограммы: целевые узлы для безусловного и условного перехода (по мере

необходимости), дерево операций, ассоциированных с данным местом в алгоритме,

представленном в исходном тексте подпрограммы

2. Реализовать модуль, формирующий граф потока управления на основе синтаксической структуры

текста подпрограмм для входных файлов

а. Программный интерфейс модуля принимает на вход коллекцию, описывающую набор

анализируемых файлов, для каждого файла – имя и соответствующее дерево разбора в виде

структуры данных, являющейся результатом работы модуля, созданного по заданию 1 (п. 3.б).

б. Результатом работы модуля является структура данных, разработанная в п. 1, содержащая

информацию о проанализированных подпрограммах и коллекция с информацией об ошибках

с. Посредством обхода дерева разбора подпрограммы, сформировать для неё граф потока

управления, порождая его узлы и формируя между ними дуги в зависимости от

синтаксической конструкции, представленной данным узлом дерева разбора: выражение,

ветвление, цикл, прерывание цикла, выход из подпрограммы – для всех синтаксических

конструкций по варианту (п. 2.b)

d. С каждым узлом графа потока управления связать дерево операций, в котором каждая

операция в составе текста программы представлена как совокупность вида операции и

соответствующих операндов (см задание 1, пп. 2.d-g)

e. При возникновении логической ошибки в синтаксической структуре при обходе дерева

разбора, сохранить в коллекции информацию об ошибке и её положении в исходном тексте

3. Реализовать тестовую программу для демонстрации работоспособности созданного модуля

a. Через аргументы командной строки программа должна принимать набор имён входных

файлов, имя выходной директории

b. Использовать модуль, разработанный в задании 1 для синтаксического анализа каждого

входного файла и формирования набора деревьев разбора

c. Использовать модуль, разработанный в п. 2 для формирования графов потока управления

каждой подпрограммы, выявленной в синтаксической структуре текстов, содержащихся во

входных файлах

d. Для каждой обнаруженной подпрограммы вывести представление графа потока управления в

отдельный файл с именем “sourceName.functionName.ext” в выходной

директории, по умолчанию размещать выходные файлы в той же

директории, что соответствующий входной

е. Для деревьев операций в графах потока управления всей совокупности подпрограмм

сформировать граф вызовов, описывающий отношения между ними в плане обращения их

друг к другу по именам и вывести его представление в дополнительный файл, по-умолчанию

размещаемый рядом с файлом, содержащим подпрограмму main.

ф. Сообщения об ошибке должны выводиться тестовой программой (не модулем, отвечающим

за анализ!) в стандартный поток вывода ошибок

4. Результаты тестирования представить в виде отчета, в который включить:

а. В части 3 привести описание разработанных структур данных

б. В части 4 описать программный интерфейс и особенности реализации разработанного модуля

с. В части 5 привести примеры исходных анализируемых текстов для всех синтаксических

конструкций разбираемого языка и соответствующие результаты разбора

Вариант 3

source: sourceItem*;

typeRef: {

|builtin: 'bool'|'byte'|'int'|'uint'|'long'|'ulong'|'char'|'string';

|custom: identifier;

|array: typeRef '(' (',')* ')';

};

funcSignature: identifier '(' list<argDef> ')' ('as' typeRef)? {

argDef: identifier ('as' typeRef)?;

};

```

sourceItem: {
|funcDef: 'function' funcSignature statement* 'end' 'function';
};
statement: {
|var: 'dim' list<identifier> 'as' typeRef; // for static typing
|if: 'if' expr 'then' statement* ('else' statement*)? 'end' 'if';
|while: 'while' expr statement* 'wend';
|do: 'do' statement* 'loop' ('while'|'until') expr;
|break: 'break';
|expression: expr ';';
};
expr: { // присваивание через '='
|binary: expr binOp expr; // где binOp - символ бинарного оператора
|unary: unOp expr; // где unOp - символ унарного оператора
|braces: '(' expr ')';
|callOrIndexer: expr '(' list<expr> ')';
|place: identifier;
|literal: bool|str|char|hex|bits|dec;
};

```

3. Описание работы

Во второй лабораторной работе будет реализован комплексный модуль для анализа потока управления в программах, написанных на разработанном языке программирования. Система строит граф потока управления (Control Flow Graph, CFG) на основе синтаксического дерева, полученного от модуля парсера первой работы.

4. Аспекты реализации

4.1. Описание структур данных

Структура CFGNode (узел графа потока):

```
struct CFGNode {  
    char *text;           // Текстовое описание операции  
    CFGNode *definitely;  // Указатель на узел безусловного перехода  
    CFGNode *conditionally; // Указатель на узел условного перехода  
    TreeNode *operationTree; // Дерево операций для этого узла  
    int id;               // Уникальный идентификатор узла  
    int printed;          // Флаг для отслеживания вывода  
    ListingNode *listingNode; // Связь с исходным кодом  
};
```

Структура FunExecution (информация о функции):

```
struct FunExecution {  
    char *name;           // Имя функции  
    char *filename;       // Исходный файл  
    TreeNode *signature;  // Сигнатура функции (тип возврата,  
    аргументы)  
    TreeNode *funCalls;   // Дерево вызовов функций  
    CFGNode *nodes;       // Граф потока управления функции  
    char **errors;        // Массив сообщений об ошибках  
    int errorsCount;      // Количество ошибок  
};
```

4.2. Описание основного алгоритма построения графа потока управления

Алгоритм кратко представлен в виде псевдокода для лучшего понимания:

(Основной алгоритм реализован в функции executionGraph)

parseResult -> executionGraph() -> FunExecution -> файлы для графического представления

INPUT: FilenameParseTree[] - массив пар (имя_файла, дерево_разбора)

OUTPUT: Array<FunExecution> - результаты анализа всех функций

ALGORITHM:

FOR EACH входной файл DO

 sourceNode - найти корневой узел AST

 sourceItems - найти все функции в корне

FOR EACH функция sourceItem DO

 funExecution - новая структура функции

 funExecution.name - имя функции

 funExecution.signature - сигнатура функции

 // Построение CFG

 funExecution.nodes - initGraph(sourceItem)

 // Анализ вызовов функций

 funExecution.funCalls - дерево вызовов

 // Сборка ошибок

 funExecution.errors - накопленные ошибки

 ADD funExecution TO результат

END FOR

END FOR

RETURN результат

4.3. Описание некоторых функций

Парсер Bison при обработке списков (функции, аргументы, операторы) создаёт бинарные деревья, где список из N элементов представлен цепью бинарных узлов. Это неудобно для обработки. Функция findListItemsUtil() линеаризует структуру:

```
Array findListItemsUtil(TreeNode *treeNode) {  
    Array items = инициализация();  
  
    TreeNode *current = treeNode;  
    while (current != NULL) {  
        if (current->childrenNumber == 1) {  
            // Один элемент списка  
            addToList(&items, current->childNodes[0]);  
            break;  
        } else if (current->childrenNumber == 2) {  
            // Рекурсивный элемент списка  
            addToList(&items, current->childNodes[0]);  
            current = current->childNodes[1]; // Переход к следующему  
        } else {  
            // Ошибка: неожиданная структура  
            addException("List structure error");  
            break;  
        }  
    }  
    return items;  
}
```

Построение графа для выражений

operationTreeNode() преобразует синтаксическое дерево выражения в граф операций, где каждая операция - узел с явными операндами.

Обработка бинарных операций

```
if (parsingTree->childrenNumber == 2) {  
    node = mallocTreeNode(parsingTree->type, NULL, 2);  
  
    if (!strcmp(parsingTree->type, "SET")) {  
        // Присваивание: левая часть - переменная  
        node->childNodes[0] = mallocTreeNode(NULL,  
                                                parsingTree->childNodes[0]->value, 0);  
        node->childNodes[1] = operationTreeNode(parsingTree->childNodes[1]);  
    } else {  
        // Арифметические операции  
        node->childNodes[0] = operationTreeNode(parsingTree->childNodes[0]);  
        node->childNodes[1] = operationTreeNode(parsingTree->childNodes[1]);  
    }  
}
```

Обработка вызовов функций

```
if (!strcmp(parsingTree->type, "callOrIndexer")) {  
    Array argsArray = findListItemsUtil(parsingTree->childNodes[1]);  
    node = mallocTreeNode("EXECUTE", NULL, argsArray.nextPosition + 1);  
  
    // Первый дочерний узел - имя функции  
    node->childNodes[0] = mallocTreeNode(NULL,  
                                            parsingTree->childNodes[0]->value, 0);  
  
    // Остальные - аргументы
```

```

for (int i = 0; i < argsArray.nextPosition; ++i) {
    node->childNodes[i + 1] =
        operationTreeNode(argsArray.elements[i]);
}

// Регистрация вызова функции для построения графа вызовов
TreeNode *callInfo = mallocTreeNode("operationTreeId",
    stringOf(node->id), 1);
callInfo->childNodes[0] = mallocTreeNode("call",
    parsingTree->childNodes[0]->value, 0);
addToList(funCalls->funCalls, callInfo);
}

```

Обработка условий

```

CFGNode *executionIfNode(TreeNode *treeNode, CFGNode *nextNode,
    CFGNode *breakNode, FunCalls *funCalls) {
    CFGNode *node = initCFGNode("");

    // Анализ структуры узла
    TreeNode *elseTreeNode = NULL;
    TreeNode *ifStatements = NULL;

    // Проверить наличие else и тела if
    if (treeNode->childrenNumber == 3) {
        // Случай: условие, then-блок, else-блок
        ifStatements = treeNode->childNodes[1];
        elseTreeNode = treeNode->childNodes[2];
    } else if (treeNode->childrenNumber == 2) {
        if (!strcmp(treeNode->childNodes[1]->type, "else")) {
            // Случай: условие и else-блок

```

```

        elseTreeNode = treeNode->childNodes[1];
    } else {
        // Случай: условие и then-блок
        ifStatements = treeNode->childNodes[1];
    }
}

```

// Создание узла условия

```
CFGNode *conditionNode =
```

```
    executionExpressionNode(treeNode->childNodes[0], funCalls);
```

// Формирование веток

```
CFGNode *thenNode = NULL;
```

```
if (ifStatements != NULL) {
```

```
    thenNode = executionNode(ifStatements, nextNode, breakNode, funCalls);
```

```
}
```

```
CFGNode *elseNode = NULL;
```

```
if (elseTreeNode != NULL) {
```

```
    elseNode = executionElseNode(elseTreeNode, nextNode, breakNode,
funCalls);
```

```
} else {
```

```
    elseNode = nextNode;
```

```
}
```

// Подключение веток к условию

```
node->definitely = conditionNode;
```

```
conditionNode->definitely = elseNode;
```

```
conditionNode->conditionally = thenNode ? thenNode : nextNode;
```

```
    return node;
}
```

Построение узлов переменных

```
CFGNode *executionVarNode(TreeNode *treeNode, CFGNode *nextNode,
                           CFGNode *breakNode) {
    CFGNode *node = initCFGNode("");
```

```
    // Извлечение типа и списка переменных
```

```
    Array variablesList = {0, 0, NULL};
```

```
    TreeNode *typeNode = NULL;
```

```
    if (treeNode->childrenNumber == 2) {
        variablesList = findListItemsUtil(treeNode->childNodes[0]);
        typeNode = treeNode->childNodes[1];
    } else {
        typeNode = treeNode->childNodes[0];
    }
}
```

```
    // Форматирование типа (включая информацию о массивах)
```

```
    char resultNodeType[1024];
```

```
    if (!strcmp(typeNode->type, "array")) {
        int arraySize = 0;
        if (typeNode->childrenNumber == 2) {
            arraySize = findListItemsUtil(typeNode->childNodes[1])
                .nextPosition;
        }
        sprintf(resultNodeType, "array of %s size %d",
            typeNode->value, arraySize);
    }
}
```

```

    } else {
        sprintf(resultNodeType, "%s", typeNode->value);
    }

    // Создание узлов присваивания для каждой переменной
    CFGNode *previous = node;
    for (int i = 0; i < variablesList.nextPosition; ++i) {
        char varText[1024];
        sprintf(varText, "AS %s %s",
            ((TreeNode *)variablesList.elements[i])->value,
            resultNodeType);

        CFGNode *initNode = initCFGNode(varText);

        // Построение дерева операций: AS (присваивание)
        TreeNode *opTree = mallocTreeNode("AS", NULL, 2);
        opTree->childNodes[0] = mallocTreeNode(NULL, resultNodeType, 0);
        opTree->childNodes[1] = mallocTreeNode(NULL,
            ((TreeNode *)variablesList.elements[i])->value, 0);

        initNode->operationTree = opTree;
        previous->definitely = initNode;
        previous = initNode;
    }

    previous->definitely = nextNode;
    return node;
}

```

Ошибки добавляются в список:

Например, неожиданная структура синтаксического дерева,
несогласованность в определении переменных.

```
void addException(char *text) {  
    char *exception = mallocString(text);  
    addToList(&exceptions, exception);  
}
```

Результаты

Созданы артефакты cfg.h - заголовочный файл с определениями структур,
cfg.c реализация анализатора потока управления, main.c тестовая
программа (дополнена из работы 1)

Пример теста для текста: (итоговый граф потока управления + дерево
операций)

```
function read_num() as int  
    dim in as int  
    dim res as int  
    res = 0;  
    while true  
        in = input();  
        if in == 13 then  
            break  
        end if  
        res = res * 10 + (in - 48);  
    wend  
    res;  
end function
```

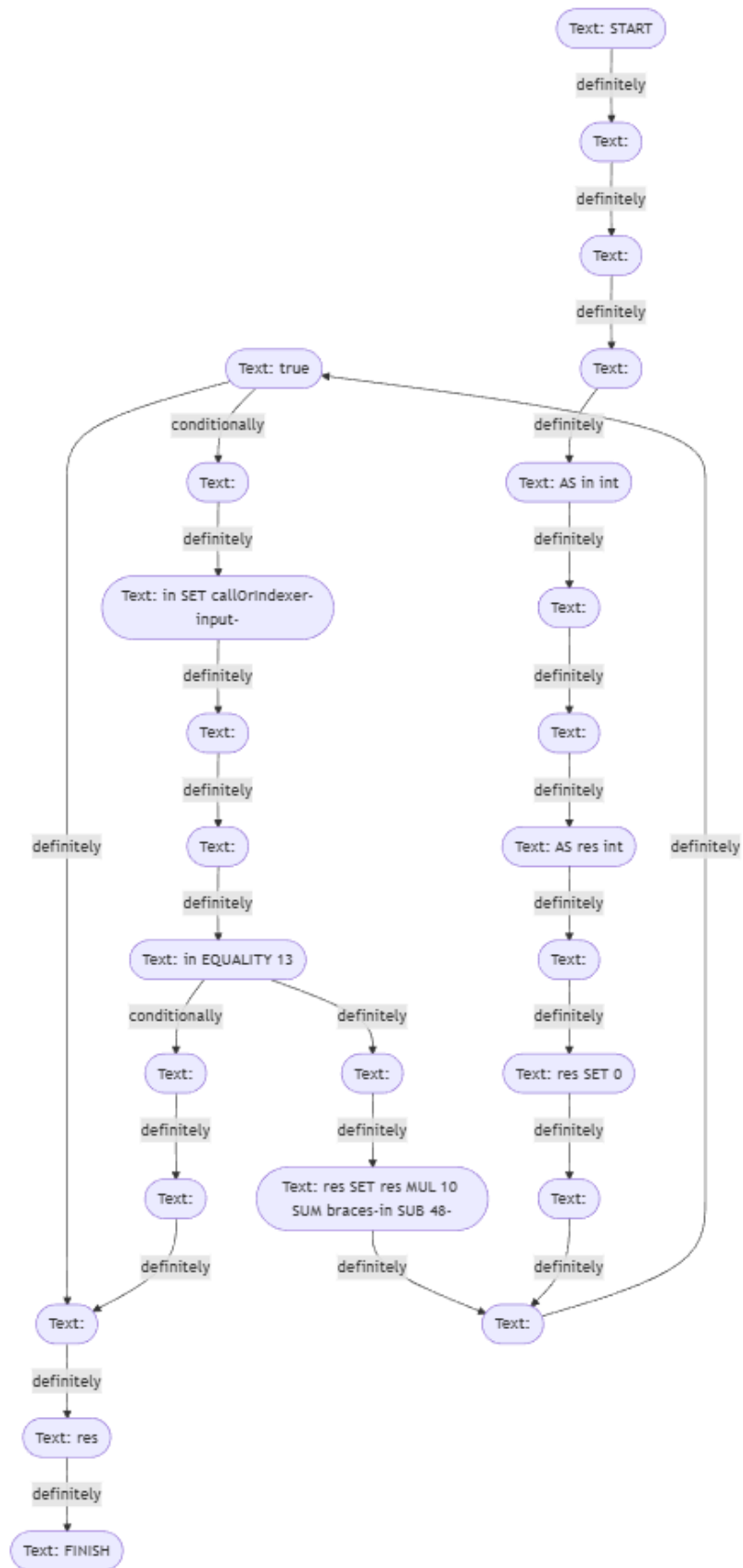


Рисунок 1 - граф потока управления

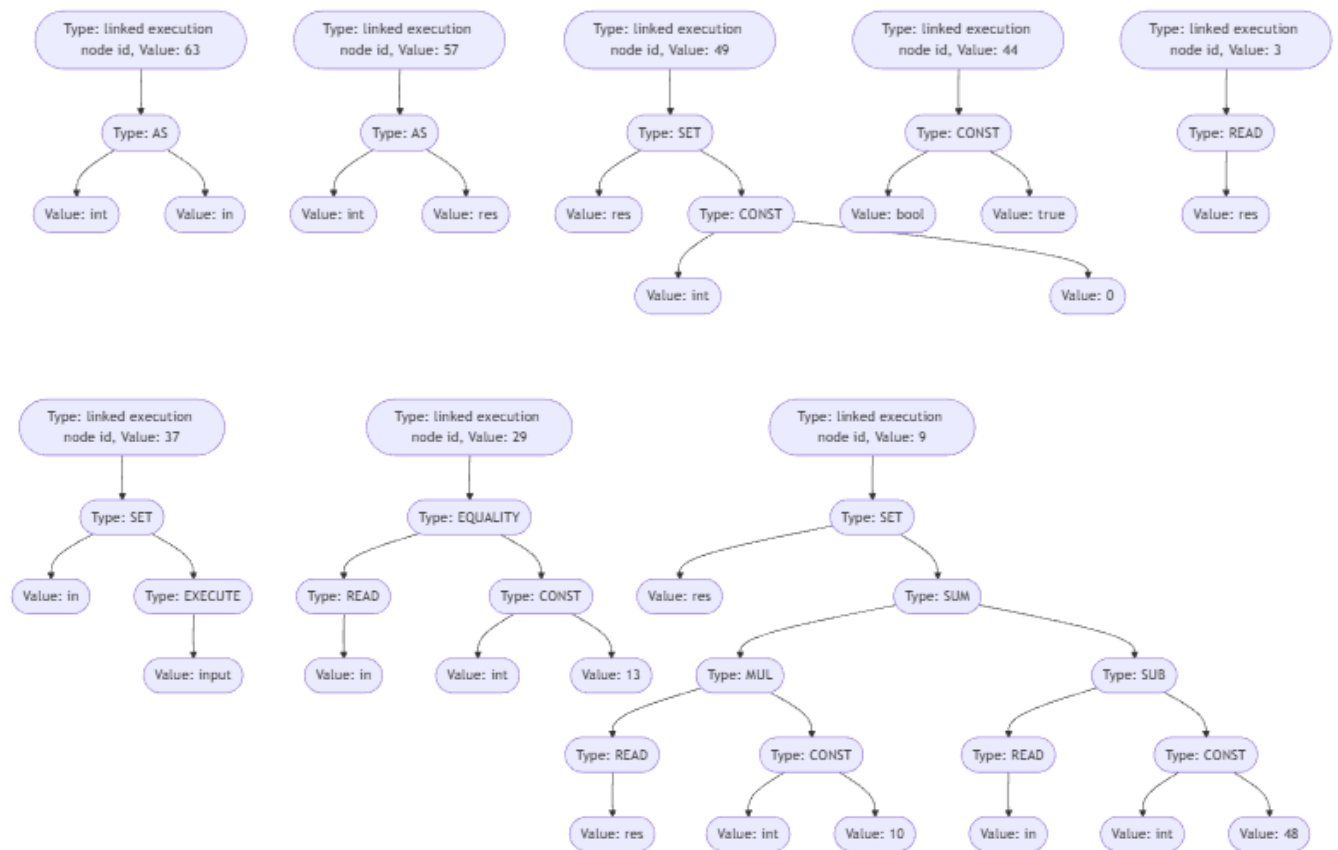


Рисунок 2 - дерево операций

Выводы

Реализованная система может быть использована как основа для следующих лабораторных работ. Лабораторная работа позволила получить глубокие знания в области графов потоков управления и практическое понимание методов анализа программ.